
CuraHarness: Online Context Curation for Automated Harness Optimization

Anonymous Author(s)

Affiliation

Address

email

Abstract

Automated harness optimization improves the code that wraps a language model into an agent through an iterative propose–evaluate loop. Recent systems such as Meta-Harness expose the full filesystem history of prior candidates, allowing a coding-agent proposer to inspect raw code, traces, and diffs instead of only compressed textual feedback. This full-history interface preserves useful evidence, but creates two scaling problems as the number of iterations grows. First, the proposer must spend more tokens rediscovering which past runs and files matter for the next edit. Second, its inspection budget does not grow with the archive: in our traces, the visible workspace expands substantially, yet the proposer still opens only a small, nearly fixed number of files per iteration. Thus, the bottleneck is not history availability, but whether limited inspection is routed to useful evidence.

We formulate this bottleneck as *online context curation*. Before each proposal, the optimizer decides which prior iterations, trace details, and workspace files should be made salient under a bounded inspection budget, with the goal of improving the cost–quality frontier of harness search. We introduce CuraHarness, a two-stage curation framework. The first stage performs iteration-level curation: it presents a compact, labelled view of prior attempts, starts from best-performing references, and widens historical exposure when optimization stalls. This stage alone improves mean held-out performance across our memory comparisons. The second stage performs file-level curation: within the curated workspace, it assigns delayed credit to files read by the proposer and prioritizes files whose inspection has correlated with productive edits. This further redirects the same bounded proposer attention and usually reduces token cost relative to the iteration-only variant, while rarely being Pareto dominated. Across three benchmarks spanning long-horizon memory and software engineering, CuraHarness improves over the uncurated full-context baseline in most benchmark–proposer settings. On the two memory benchmarks, it opens lower-cost operating points on the cost–quality frontier. On SWE-bench Verified, the same curation principle trades a modest cost increase for a much stronger harness, raising full-set passrate from 0.440 for the source harness and 0.458 for full-context optimization to 0.640. These results show that proposer context is not a static prompt artifact: it is an online optimization variable that controls both where tokens are spent and which evidence drives the next edit.

1 Introduction

Agent *harnesses*—the code that wraps a large language model (LLM) into a working agent—strongly shape downstream performance. Agents built on the same LLM but with different harnesses can differ by up to $6\times$ on the same benchmark [1]. This has motivated *automated harness optimization*: an

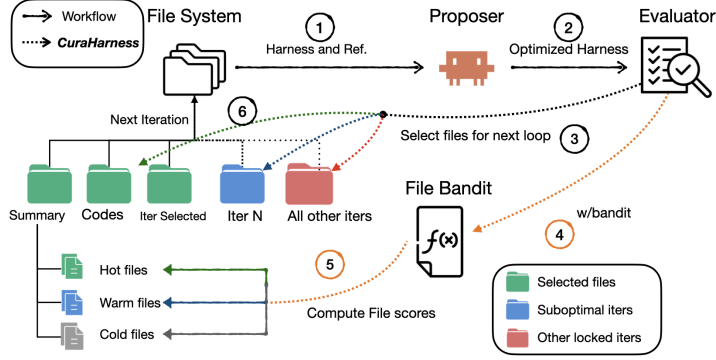


Figure 1: CuraHarness inserts an online context-curation loop into optimization. Solid arrows show the standard propose–evaluate workflow: the proposer edits the exposed harness context and the evaluator scores the resulting candidate. Dotted arrows show our contribution: evaluation feedback updates file utilities, ranks files into hot/warm/cold tiers, and selects which reference iterations and files are exposed in the next filesystem view.

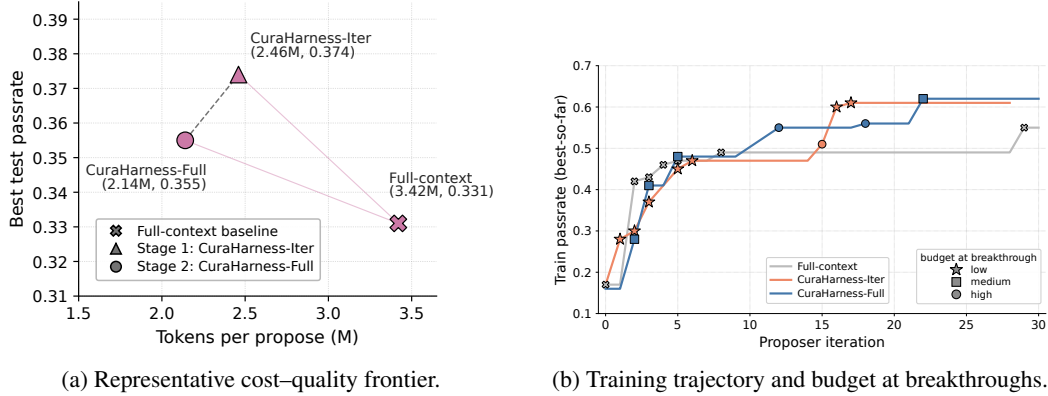


Figure 2: Cost–quality frontier on LoCoMo/codex54 and optimization dynamics on Long-MemEval/claudekimi. Left: CURAHARNES-ITER improves best held-out passrate while reducing token cost relative to full-context, and CURAHARNES-FULL selects a lower-cost operating point that retains most of the gain. Right: best-so-far training passrate over proposer iterations; markers denote the active budget when a new best candidate is found. Breakthroughs occur under multiple budget levels, supporting an adaptive historical-scope schedule.

outer propose–evaluate loop in which an LLM-driven proposer suggests candidate agents, prompts, or memory components, and an evaluator scores each candidate to guide later proposals [2, 3, 4, 5, 6].

A central design question is how much prior optimization history to show the proposer. Many prior systems compress feedback into scalar scores, short summaries, or the latest candidate [7, 4, 8, 3, 5, 9, 10, 11]. This lowers cost, but discards the raw code, traces, and diffs needed to connect failures to specific harness decisions. Meta-Harness takes the opposite approach: it gives a coding-agent proposer access to the full revision history as a filesystem [6]. This preserves useful evidence, but does not remove the cost of finding it. As the search horizon grows, the proposer must rediscover which past runs and files matter for the next edit, while its inspection budget remains limited.

Our traces suggest that the main bottleneck is not history availability, but attention allocation. The exposed workspace can grow substantially over the course of optimization, yet the proposer still opens only a small and nearly fixed number of files per iteration. Thus, the question is not simply whether the full history is present, but whether the proposer’s bounded inspection effort is routed to useful evidence. We therefore formulate automated harness optimization as an *online context*

curation problem. Before each proposal, the optimizer decides which prior iterations, trace details, and workspace files should be made salient under a bounded inspection budget. The goal is to improve the cost–quality frontier of harness search: to obtain stronger harnesses at a given proposal cost, or to reach a target quality with less unproductive inspection. We introduce CuraHarness, a two-stage curation framework. The first stage performs iteration-level curation. It presents a compact, labelled view of prior attempts, starts from strong reference candidates, and widens historical exposure when optimization stalls. This preserves direct access to code and traces while avoiding the assumption that every past iteration is equally relevant. The second stage performs file-level curation. Within the curated workspace, CuraHarness assigns delayed credit to files read by the proposer and prioritizes files whose inspection has correlated with productive edits. Together, the two stages route the same bounded proposer attention at different granularities: iteration-level curation selects which parts of the search history enter the workspace, while file-level curation guides which files are most worth inspecting inside it.

This framing makes clear that CuraHarness is not only a token-reduction method. On compact memory-harness benchmarks, curated evidence can replace exploratory tool use, reducing cost while maintaining or improving held-out quality. On larger software-engineering harnesses, the same principle can instead induce deeper source and trace comparison, increasing cost but producing a stronger harness. Both cases are consistent with the same objective: moving optimization to a better point on the cost–quality frontier. Figure 2 illustrates this: curation can open lower-cost memory-benchmark operating points, while adaptive historical scope remains useful because improvements arise under multiple budget levels.

We evaluate the iteration-only variant and the full two-stage CuraHarness system on SWE-bench [12], LongMemEval [13], and LoCoMo [14]. Across the two memory benchmarks, iteration-level curation improves mean held-out performance, showing that historical scope affects how the proposer uses prior experience. File-level curation further changes the operating point, usually reducing token cost relative to the iteration-only variant while rarely being Pareto dominated. Across three benchmarks, CuraHarness improves over the uncurated full-context baseline in most benchmark–proposer settings. On the memory benchmarks, it opens lower-cost operating points on the cost–quality frontier. On SWE-bench Verified, it trades a modest cost increase for a much stronger harness, raising full-set passrate from 0.440 for the source harness and 0.458 for full-context optimization to 0.640. These results show that proposer context is not a static prompt artifact, but an online optimization variable that controls where tokens are spent and which evidence drives the next edit.

2 Related Work

Automated harness optimization. The closest line of work uses LLMs as outer-loop optimizers for prompts, agent programs, or harness code. OPRO [15] casts prompt engineering as black-box optimization with LLM-generated candidates; GEPA [5] refines prompts using reflective textual feedback. ADAS [7] moves from prompts to agent programs, having an LLM propose new scaffolds from an archive of prior designs. Meta-Harness [6] is closest to our setting: it optimizes harness files with a coding agent that can inspect the full filesystem history of previous candidates. Related program-search systems such as FunSearch, AlphaEvolve, and OpenEvolve use LLMs as mutators under automated fitness functions; we discuss these broader connections in Appendix ?? . CuraHarness keeps this file-backed optimization interface but changes the information policy around it. Rather than exposing the entire accumulated workspace or compressing it into a fixed summary, CuraHarness chooses which prior iterations and file hints should be salient before each proposal.

Context curation in iterative optimization. Our motivation aligns with evidence that more context is not always better. Liu et al. [16] show that models use long contexts unevenly; CODEFILTER [17] demonstrates that filtering retrieved code context can outperform passing all retrieved chunks. Nie et al. [18] further show that iterative generative optimization is sensitive to design choices such as credit horizon and experience batching. CuraHarness carries these ideas into automated harness optimization by treating proposer attention as an online control variable: a progressive iteration policy manages history scope, and delayed utility estimates manage file emphasis.

3 Problem Statement

Harness optimization. A harness is a stateful program that wraps a fixed language model M and determines what context M sees at each step. For a harness H and task instance $x \sim \mathcal{X}$, we execute a rollout trajectory $\tau \sim p_M(H, x)$ and score it with task-specific reward $r(\tau, x)$. The goal of automated harness optimization is to find

$$H^* = \arg \max_H \mathbb{E}_{x \sim \mathcal{X}, \tau \sim p_M(H, x)} [r(\tau, x)]. \quad (1)$$

Search loop with context curation. A run of the outer loop produces a sequence of evaluated candidates $\{(H_i, E_i)\}_{i=1}^t$, where E_i bundles H_i 's source code, scores, and execution traces. We refer to the accumulated archive $D_t = \{(H_i, E_i)\}_{i \leq t}$ as the experience pool. At each iteration t , the proposer $\mathcal{P}$ generates a new candidate conditioned on a curated context $C_t \subseteq D_t$:

$$H_{t+1} \sim P(\cdot \mid C_t), \quad (2)$$

where $C_t = \mathcal{C}(D_t, s_t)$ is produced by a context curation policy $\mathcal{C}$ that selects which subset of D_t to surface, and s_t denotes the proposer's auxiliary state at iteration t (e.g., current frontier, recent regressions). After evaluation, $D_{t+1} = D_t \cup \{(H_{t+1}, E_{t+1})\}$. This formulation generalizes common context interfaces: full-history filesystem [6] access corresponds to rendering the entire archive to the proposer, while static feedback compression [15, 5] corresponds to using a fixed summarizer $C_t = \phi(D_t)$.

4 Method

This section describes CuraHarness, our framework for online context curation in automated harness optimization. At each iteration, CuraHarness selects a curation tier based on the current optimization state—whether the last proposal improved the frontier or the search has stalled—and curates context along two complementary axes: an iteration-level strategy chooses which prior rounds' artifacts to expose, and a file-level strategy chooses which files within those rounds the proposer actually sees. The two operate jointly: iteration-level selection defines the pool over which file-level selection acts.

4.1 Curation Tier

We organize curation into three tiers—*low*, *medium*, and *high*—that control how many prior iterations and files are made accessible to the proposer. The tier at iteration t is determined by whether the search is currently making progress. Let $J(E)$ denote the scalar training score used to rank evaluated candidates, as defined in Sec. 3. We define the frontier-improvement signal as

$$\rho_{t+1} = J(E_{t+1}) - \max_{(H_i, E_i) \in D_t} J(E_i). \quad (3)$$

A positive value means that the newly proposed harness improves over all previous candidates in the current run. The stagnation count is updated by

$$z_{t+1} = \begin{cases} 0, & \rho_{t+1} > 0, \\ z_t + 1, & \rho_{t+1} \leq 0. \end{cases} \quad (4)$$

The curation tier is then selected from the stagnation count:

$$\ell_t = \begin{cases} \text{low}, & z_t = 0, \\ \text{medium}, & 0 < z_t < K_{\text{stall}}, \\ \text{high}, & z_t \geq K_{\text{stall}}, \end{cases} \quad (5)$$

where K_{stall} is the number of consecutive non-improving iterations required before opening the widest historical view.

4.2 Iteration-level curation

The iteration-level controller determines the historical scope of the evidence workspace. Its goal is to adapt the amount of prior evidence to the search state. After a frontier-improving proposal,

138 a compact context centered on the successful direction is usually sufficient. After repeated non-
 139 improving proposals, the proposer benefits from a broader view of earlier edits, regressions, traces, and
 140 alternative candidates. Each tier controls two workspace components: labelled reference iterations and
 141 unresolved failure traces. The reference scope follows a 1/3/all schedule: `low` exposes one reference
 142 iteration, `medium` exposes three, and `high` exposes all prior iterations. The trace scope follows a
 143 10/30/all schedule. Trace selection is failure-driven: cases that previous candidates still failed to
 144 solve are surfaced before easier or already-solved cases, since unresolved cases reveal persistent
 145 weaknesses in the current harness. These two evidence types serve different roles. Reference
 146 iterations expose concrete prior harness states, edits, and diffs. Failure traces identify task instances
 147 that remained unsolved after previous optimization attempts. When the run is making progress, a
 148 small reference set and a compact unresolved-failure set support local refinement. When progress
 149 stalls, the controller widens the scope so that the proposer can compare more prior edits and avoid
 150 repeating plausible but exhausted changes. The tier is not a hard cap on proposer computation. A
 151 higher tier exposes more historical evidence, but the proposer still decides which files to inspect, and
 152 its observed read volume remains far smaller than the available workspace. Iteration-level curation
 153 therefore routes limited inspection effort toward more useful historical evidence rather than enforcing
 154 a strict external budget. The iteration-only variant, CURAHARNESS-ITER, uses this controller without
 155 the file-level utility model.

156 4.3 File-level delayed-credit curation

157 The file-level controller operates within the workspace W_t selected by the iteration-level stage. It
 158 does not choose which prior iterations enter the workspace; instead, it ranks readable files in that
 159 workspace so that the proposer sees a small set of hot and warm files rather than an undifferentiated
 160 filesystem tree. The full two-stage system, CURAHARNESS-FULL, combines iteration-level curation
 161 with this file-level delayed-credit model.

162 Let $\mathcal{A}_t$ denote the files read by the proposer when constructing H_{t+1} . Since a file’s usefulness is
 163 observed only after evaluation, credit is assigned with delay. For each file f , the curator stores

$$B_t(f) = (n_t(f), r_t(f)), \quad (6)$$

164 where $n_t(f)$ is the number of past read events and $r_t(f)$ is the cumulative reward assigned to those
 165 reads. After observing ρ_{t+1} , the statistics are updated by

$$n_{t+1}(f) = n_t(f) + \mathbf{1}[f \in \mathcal{A}_t], \quad (7)$$

$$r_{t+1}(f) = r_t(f) + \rho_{t+1} \mathbf{1}[f \in \mathcal{A}_t]. \quad (8)$$

166 This is shared delayed credit, not causal attribution: files read before a frontier-improving proposal
 167 receive positive credit, while files read before a non-improving proposal receive non-positive credit.
 168 We therefore use the signal only as a routing prior.

169 At the next iteration, each readable non-core file is scored as

$$u_t(f) = \frac{r_t(f)}{n_t(f) + \epsilon} + \alpha \sqrt{\frac{\log(1 + \eta_t)}{n_t(f) + \epsilon}} - \beta \hat{c}(f), \quad (9)$$

170 where $\eta_t = \sum_{f'} n_t(f')$, $\hat{c}(f)$ estimates read cost from file size or token count, α controls exploration,
 171 β controls cost sensitivity, and $\epsilon > 0$ smooths cold-start files. The three terms capture, respectively,
 172 past reward correlation, exploration of rarely read files, and a penalty for expensive reads.

173 Core files needed to understand or edit the harness are pinned. The remaining files are ranked by
 174 $u_t(f)$ and assigned to hot, warm, or cold tiers. Hot files are surfaced most prominently, warm files
 175 are secondary hints, and cold files are not highlighted although they may remain reachable depending
 176 on the workspace tier. Thus, iteration-level curation determines which historical evidence is available,
 177 while file-level curation determines which files within that evidence should receive the proposer’s
 178 limited inspection effort.

179 In Sec. 5, CURAHARNESS-ITER denotes the iteration-only controller, while CURAHARNESS-FULL
 180 denotes the two-stage system with file-level delayed-credit curation.

Table 1: Main results across memory and SWE-bench harnesses. Memory numbers are averaged over three 30-iteration runs; SWE-bench is one 20-iteration *claudekimi* run. “Best” reports the best held-out or Verified passrate. Cost is millions of tokens per proposal, with Δ relative to full-context.

Benchmark	Proposer	Policy	Runs / iters	Train $\pm$ std	Test mean $\pm$ std	Best / verified	Tokens / prop.	Δ
LoCoMo	claudekimi	full-context	3 $\times$ 30	39.2% $\pm$ 0.019	33.1% $\pm$ 0.015	34.2%	3.42M	—
		CuraHarness-Iter	3 $\times$ 30	41.2% $\pm$ 0.022	35.5% $\pm$ 0.021	37.3%	1.86M	−46%
		CuraHarness-Full	3 $\times$ 30	42.1% $\pm$ 0.029	35.5% $\pm$ 0.008	36.2%	2.63M	−23%
LoCoMo	codex54	full-context	3 $\times$ 30	41.1% $\pm$ 0.019	33.1% $\pm$ 0.013	34.0%	3.42M	—
		CuraHarness-Iter	3 $\times$ 30	41.7% $\pm$ 0.014	37.4% $\pm$ 0.014	38.8%	2.46M	−28%
		CuraHarness-Full	3 $\times$ 30	40.8% $\pm$ 0.019	35.5% $\pm$ 0.027	38.7%	2.14M	−37%
LongMemEval	claudekimi	full-context	3 $\times$ 30	58.7% $\pm$ 0.055	49.8% $\pm$ 0.030	53.0%	3.71M	—
		CuraHarness-Iter	3 $\times$ 30	60.3% $\pm$ 0.006	50.5% $\pm$ 0.013	52.0%	3.37M	−9%
		CuraHarness-Full	3 $\times$ 30	58.0% $\pm$ 0.036	51.0% $\pm$ 0.038	54.5%	2.55M	−31%
LongMemEval	codex54	full-context	3 $\times$ 30	56.0% $\pm$ 0.030	49.7% $\pm$ 0.010	50.8%	2.93M	—
		CuraHarness-Iter	3 $\times$ 30	56.7% $\pm$ 0.038	49.9% $\pm$ 0.028	52.8%	2.66M	−9%
		CuraHarness-Full	3 $\times$ 30	52.0% $\pm$ 0.030	46.3% $\pm$ 0.010	47.3%	2.18M	−26%
SWE-bench	claudekimi	full-context	1 $\times$ 20	50.0%	—	45.8%	3.22M	—
		CuraHarness-Iter	1 $\times$ 20	53.3%	—	62.0%	3.78M	+17%
		CuraHarness-Full	1 $\times$ 20	53.3%	—	64.0%	3.51M	+9%

5 Experiments

We evaluate whether context curation improves automated harness optimization by changing what the proposer inspects under a bounded search budget. Our experiments separate two variants of CuraHarness. CURAHARNESS-ITER applies only iteration-level curation: it selects a compact set of prior iterations and labels them by optimization role. CURAHARNESS-FULL adds file-level curation on top of the same iteration controller, using delayed utility signals to prioritize files inside the selected workspace.

5.1 Setup

Benchmarks. We use two long-horizon memory benchmarks and one software-engineering benchmark. **LoCoMo** [14] probes multi-session conversation memory; we optimize on 80 training examples and report on the full 1,449-example split. **LongMemEval** [13] probes retrieval, temporal reasoning, and consolidation over long histories; we optimize on 100 training examples and report on the 400-example test split. **SWE-bench Verified** [19] (abbreviated as SWE-bench) evaluates multi-file coding-agent capabilities; we optimize on `trainfirst30` and re-evaluate selected candidates on the full 500-issue verified set.

Harnesses and proposers. On the memory benchmarks the inner harness is a MemGPT-style memory agent using Qwen3-8B [20]. On SWE-bench the inner solver is a fixed Mini-SWE agent driven by DeepSeek v4 Flash, which isolates harness-curation effects from solver changes. We instantiate the outer proposer with two coding agent scaffolds: a Codex CLI proposer with GPT-5.4 (*codex54*) and a Claude Code-compatible proposer with Kimi K2.6 (*claudekimi*). Both run on the memory benchmarks; SWE-bench uses *claudekimi* only.

5.2 Main Results

Table 1 reports the cost–quality results across the two memory benchmarks and SWE-bench Verified. CURAHARNESS-ITER isolates iteration-level curation: it exposes labelled reference iterations under the `low`→`medium`→`high` budget schedule, without using file-level utility. CURAHARNESS-FULL adds file-level curation on top of the same historical-scope controller, using delayed file utilities to prioritize evidence within the selected workspace.

Iteration-level curation improves memory performance. On the memory benchmarks, CURAHARNESS-ITER improves mean held-out passrate over full-context in all four benchmark–proposer cells. The gains are +2.4 pp on LoCoMo/*claudekimi*, +4.3 pp on LoCoMo/*codex54*, +0.7 pp on LongMemEval/*claudekimi*, and +0.2 pp on LongMemEval/*codex54*. It also reduces per-proposal cost in all four memory cells, by 9–46% relative to full-context. The LoCoMo improvements are larger than the run-to-run variation, while the LongMemEval gains are smaller and should

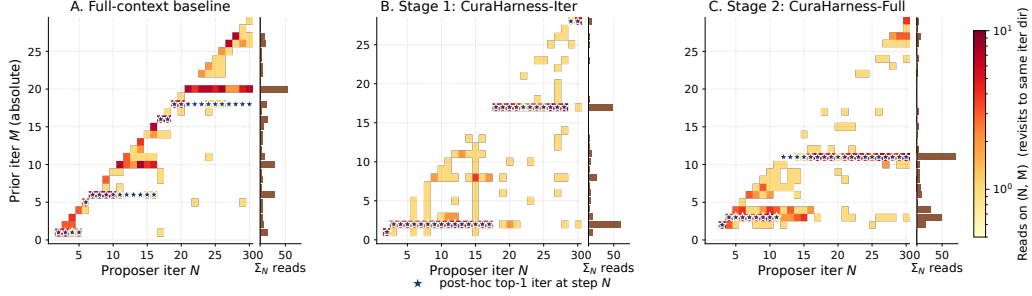


Figure 3: Reference-iteration attention on LoCoMo. Each cell counts Read calls from proposer iteration N to prior iteration M ; darker cells indicate more reads, and blue stars mark the current top-1 prior iteration. Full-context shows a recency-biased diagonal pattern, while CuraHarness redirects reads toward high-performing prior iterations.

be interpreted as directional. These results show that controlling historical scope alone can improve the memory cost–quality frontier.

File-level curation expands the observed frontier, but does not universally dominate. On the memory benchmarks, CURAHARNES-FULL is cheaper than full-context in all four cells, reducing per-proposal cost by 23–37%. It also improves both cost and best held-out passrate over full-context in three of the four cells. The exception is LongMemEval/codex54, where CURAHARNES-FULL lowers cost by 26% but decreases best held-out passrate from 50.8% to 47.3%. Thus, file-level curation should be read as an operating-point selector: it can produce a Pareto improvement when its file prior is well aligned, but it can also trade quality for lower cost.

SWE-bench shows that curation is not only token reduction. The SWE-bench Verified experiment uses a single 20-iteration claudekimi run, so we treat its absolute gap as single-run evidence rather than a variance-estimated effect. In this setting, the curated policies spend more tokens per proposal than full-context, but achieve substantially higher verified passrate. Full-context reaches 45.8%, while CURAHARNES-ITER reaches 62.0% and CURAHARNES-FULL reaches 64.0%. Token cost increases from 3.22M per proposal for full-context to 3.78M for CURAHARNES-ITER and 3.51M for CURAHARNES-FULL. This result indicates that curation optimizes frontier position rather than token count alone: it can reduce unproductive inspection in compact memory harnesses, or spend additional inspection to obtain stronger software-engineering harnesses.

Together, these results motivate the analyses below. Analysis I studies how curation reallocates reference-iteration reads, Analysis II examines how file-level utility shifts the operating point, and Analysis III explains why the token direction differs between memory and software-engineering benchmarks.

5.3 Analysis I: Curation redirects bounded inspection

We first examine how the proposer uses history under a bounded inspection budget. In the full-context baseline, the visible history grows with the optimization horizon, but the proposer does not inspect it proportionally. At the end of a run, the available reference-iteration pool expands from roughly 2 to 27 directories, while the proposer touches only a few of them per proposal; full bin-level statistics are in Appendix A.5.

Benchmark	Avail. dirs	Touched dirs	Ref. reads	Touch rate
LongMemEval	2.0→27.0	0.9→3.5	4.5→7.4	45→13%
LoCoMo	2.0→27.0	1.7→4.7	4.5→10.5	85→17%

Figure 3 shows where this limited inspection goes. Full-context reads are strongly recency-biased: when proposing iteration N , the agent mostly inspects nearby prior iterations. CURAHARNES-ITER redirects those reads toward labelled high-performing references. It raises the top-1 reference-read share from 52.1% to 77.7% on LongMemEval and from 31.1% to 53.8% on LoCoMo; on LoCoMo, recent-3 share drops from 59.4% to 36.3%, showing that curation reallocates reads rather than simply adding more context.

Thus, the benefit of iteration-level curation is not exhaustive history inspection. It is better routing of a nearly fixed inspection budget toward stronger historical anchors. The adaptive budget schedule keeps this routing compact by default and widens history only when stalled proposals need broader comparisons.

5.4 Analysis II: File-level curation selects the evidence channel

Iteration-level and file-level curation route the same bounded proposer budget through different evidence channels. Iteration-level curation supplies a *history prior*: labelled prior candidates, diffs, and traces. File-level curation supplies a *file prior*: source and trace files whose previous inspection received positive delayed utility. Adding the file axis can therefore shift the proposer from explicit history comparison toward file-local exploitation.

A force-budget probe makes this channel switch visible. With the same bandit policy, forcing low nearly closes the reference-history channel, whereas forcing high reopens it. The quality effect differs by benchmark, indicating that the value of history is conditional rather than universal; full results are in Appendix A.

Benchmark	Forced budget	Ref-read share	Train passrate
LoCoMo	low	0.1%	0.3875
LoCoMo	high	36.4%	0.4000
LongMemEval	low	0.1%	0.6300
LongMemEval	high	36.4%	0.5800

This explains why CURAHARNESS-FULL is best understood as an operating-point selector, not as a uniformly additive module. When useful edits are localized in stable high-utility files, file-level curation can improve cost and quality, as on LongMemEval/claudekimi. When progress depends on broader historical contrast, the same file prior may over-concentrate attention, producing a cheaper but weaker point, as on LongMemEval/codex54. File-level curation therefore expands the cost-quality frontier by selecting how much the optimizer should exploit learned file-local evidence.

5.5 Analysis III: Benchmark structure determines token direction

Finally, we analyze why curation reduces tokens on memory harnesses but increases them on SWE-bench. The key difference is how redirected attention changes the proposer trajectory.

Cell	Tool turns	Cache/turn	Lines/turn
LongMemEval	50.2→38.6 (−23%)	72.4k→52.0k (−28%)	flat
SWE-mini	63.5→73.3 (+16%)	73.0k→66.5k (−9%)	52.2→65.6 (+26%)

On LongMemEval, curation shortens the trajectory: the proposer needs fewer tool turns and carries a smaller cache footprint per turn. In compact memory harnesses, curated evidence often substitutes for exploratory browsing. On SWE-bench, the opposite happens. Although cache per turn decreases, the proposer takes more tool turns and reads more lines per turn, because the curated history points it toward deeper source and trace comparisons inside a multi-file agent control loop.

CuraHarness should therefore be viewed as a cost-quality frontier method rather than a pure token-reduction method. On memory harnesses, better attention routing often lowers cost; on software-engineering harnesses, the same routing can spend more tokens to obtain a stronger harness.

5.6 Ablations

We ablate three design choices in CuraHarness: bounded historical exposure, best/worst iteration labels, and the reference-budget schedule. Table 2 summarizes the main ablation results.

First, bounded history is already a strong intervention. Random@3 improves over full-context despite using no score labels, file utilities, or adaptive budgeting. CuraHarness-Iter improves further, suggesting that the gain comes not only from shrinking the archive, but also from making useful historical evidence salient.

Second, the useful label is mainly the positive one. Across proposer traces, best-tagged reference iterations attract substantially more reads than unmarked history, while worst-tagged iterations are

Table 2: Ablations separating bounded historical exposure and budgeted reference-history access. Top: LoCoMo/codex54 test results over three 30-iteration runs. Bottom: claudekimi memory force-budget runs, reported on train because they diagnose budget behavior rather than final model selection.

Cell	Policy	Score	Best test	Tokens / prop.	Ref. share
<i>Iter-axis selection ablation (LoCoMo / codex54)</i>					
bounded history	full-context	0.331 ± 0.013	0.340	3.42M	—
	Random@3	0.350 ± 0.027	0.381	2.66M	—
labelled budget	CuraHarness-Iter	0.374 ± 0.014	0.388	2.46M	—
	CuraHarness-Full	0.355 ± 0.027	0.387	2.14M	—
<i>Force-budget ablation (claudekimi memory, train)</i>					
LoCoMo	CuraHarness-Full force-high	0.4000	—	4.21M	36.4%
	CuraHarness-Full force-low	0.3875	—	2.79M	0.1%
LongMemEval	CuraHarness-Full force-high	0.5800	—	3.73M	36.4%
	CuraHarness-Full force-low	0.6300	—	2.83M	0.1%

inspected much less reliably. The proposer therefore uses best references as anchors for the next edit, but does not consistently treat worst references as negative examples.

Third, the reference budget should stay small by default and expand only when needed. The force-budget rows show that force-low nearly closes the reference-history channel, while force-high preserves it at higher token cost. The dynamic schedule therefore uses low budget as a focus mechanism and escalates only when broader historical evidence is likely to help.

6 Limitations and Conclusion

CuraHarness improves automated harness optimization by treating proposer-visible context as an online control variable rather than a static full-history workspace. Its iteration-level controller selects salient prior candidates and traces, while its file-level controller prioritizes files whose inspection has previously correlated with frontier improvements. Across memory and SWE-bench settings, this redirects the proposer’s limited inspection effort and improves the cost–quality frontier: on compact memory harnesses it often reduces exploratory tool use and token cost, while on SWE-bench it spends more tokens on deeper source–trace comparison and yields stronger final quality. However, the proposer remains a black-box coding agent whose internal policy cannot be controlled; our inspection budget is behavioral rather than a hard cap on tool calls or tokens; and empirical coverage is limited, especially for SWE-bench, which is based on a single 20-iteration run. Broader validation across proposers, benchmarks, and explicit token/tool-budget controllers remains future work.

References

- [1] Muxin Tian, Zhe Wang, Blair Yang, Zhenwei Tang, Kunlun Zhu, Honghua Dong, Hanchen Li, Xinni Xie, Guangjing Wang, and Jiaxuan You. Swe-bench mobile: Can large language model agents develop industry-level mobile applications? *arXiv preprint arXiv:2602.09540*, 2026.
- [2] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.
- [3] Alexander Novikov, Ngân V u, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- [4] Asankhaya Sharma. OpenEvolve: Open-source implementation of AlphaEvolve. <https://github.com/codelion/openevolve>, 2025.

- [5] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. Gega: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.
- [6] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.
- [7] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.
- [8] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*, 2026.
- [9] Yoonho Lee, Joseph Boen, and Chelsea Finn. Feedback descent: Open-ended text optimization via pairwise comparison. *arXiv preprint arXiv:2511.07919*, 2025.
- [10] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.
- [11] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*, 2023.
- [12] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024.
- [13] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Long-MemEval: Benchmarking chat assistants on long-term interactive memory. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [14] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, 2024.
- [15] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*, 2024.
- [16] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [17] Yanzhou Li, Shangqing Liu, Kangjie Chen, Tianwei Zhang, and Yang Liu. Impact-driven context filtering for cross-file code completion. In *Conference on Language Modeling*, 2025.
- [18] Allen Nie, Xavier Daull, Zhiyi Kuang, Abhinav Akkiraju, Anish Chaudhuri, Max Piasevoli, Ryan Rong, YuCheng Yuan, Prerit Choudhary, Shannon Xiao, Rasool Fakoor, Adith Swaminathan, and Ching-An Cheng. Understanding the challenges in iterative generative optimization with LLMs. *arXiv preprint arXiv:2603.23994*, 2026.
- [19] OpenAI. Introducing SWE-bench Verified. <https://openai.com/index/introducing-swe-bench-verified/>, August 2024. Accessed: 2026-05-04.
- [20] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

A Technical appendices and supplementary material

Our code is available at <https://anonymous.4open.science/r/CuraHarness-36ED/>.

A Ablation Details

This appendix provides the ablation results summarized in Section 5.6. We separate four questions. First, does simply bounding historical exposure help, even without labels or file-level utility? Second, do best and worst reference labels affect proposer behavior symmetrically? Third, what low/medium reference budget should the iteration-level controller use? Finally, does full-context access create more usable attention, or merely a larger visible archive?

A.1 Bounded historical exposure

Table ?? isolates the effect of bounding the reference-history axis on LoCoMo/codex54. Random@3 exposes three prior iterations without score labels, file utility, or adaptive budgeting. It therefore tests whether reducing the unstructured archive is useful by itself.

Random@3 improves over full-context while reducing token cost, showing that the full archive can distract the proposer even when all historical evidence is available. CuraHarness-Iter improves further, indicating that labelled selection and adaptive scope add value beyond simply limiting the number of visible iterations. CuraHarness-Full obtains the lowest token cost in this cell, but with a lower mean test score than CuraHarness-Iter; this reflects the cost-quality trade-off introduced by the more aggressive file-level controller.

A.2 Best versus worst reference labels

CuraHarness can expose both best and worst reference iterations. This ablation asks whether the proposer uses the two labels symmetrically. We measure Read calls into labelled reference-iteration directories on retained LongMemEval/kimi CuraHarness-Full runs. A slot is one available reference-iteration directory under a given label at prompt-build time. Normalizing by slots removes the confound that different labels may expose different numbers of directories.

Table 3 shows that best-tagged directories receive substantially more attention than other reference directories. Across the three runs, best references receive roughly 14–22× more Read calls per available slot than unmarked references. Worst-tagged directories are read at roughly the same rate as, or below, unmarked references, and receive essentially zero line volume. Thus, the proposer treats best references as positive implementation anchors, but does not reliably use worst references as negative examples.

A.3 Low/medium best-reference budget grid

Table 4 ablates the positive-reference budget used by CuraHarness-Iter, without worst references. Each setting is a single 30-iteration LoCoMo/kimi run on the 80-example training split. The notation $1LmM$ means that the low tier exposes L best references and the medium tier exposes up to M best references.

The best setting is 11m3: one best reference at low budget and up to three references at medium budget. It obtains both the highest best training passrate and the highest average score. Increasing the low-budget reference count to two or three hurts performance, and increasing the medium budget to seven is consistently worse. This supports the schedule used in CuraHarness: low budget should provide one clear anchor, while medium budget should allow a small comparison set without overwhelming the proposer’s bounded reference attention.

A.4 Forced budget tiers

The adaptive budget policy changes tier during a run, so budget level and optimization phase are correlated. To isolate the behavior of the budget tier itself, Table 5 reports force-low and force-high CuraHarness-Full runs on the claudekimi memory setting. These runs are reported on train because they diagnose budget behavior rather than final held-out model selection.

Table 3: Best/worst reference-label read distribution on retained LongMemEval/kimi CuraHarness-Full runs. Reads are normalized by the number of available reference-iteration slots under each label.

Run	Bucket	Slots	Reads	Reads / slot	Lines / slot
bandit_v3 r1	best-iter dirs	63	231	3.67	258.9
	worst-iter dir	24	3	0.12	0.0
	other ref dirs	131	22	0.17	4.8
bandit_v3 r2	best-iter dirs	74	200	2.70	180.1
	worst-iter dir	18	4	0.22	0.0
	other ref dirs	166	21	0.13	10.5
bandit_v4 r1	best-iter dirs	71	245	3.45	133.9
	worst-iter dir	18	8	0.44	0.0
	other ref dirs	183	45	0.25	9.1

Table 4: Low/medium reference-budget ablation for CuraHarness-Iter without worst references on LoCoMo with the kimi proposer. Each setting is a single 30-iteration run on the 80-example training split. $1LmM$ exposes L best references at low budget and up to M references at medium budget.

Setting	Best passrate	Avg. score	Avg. tokens	Top-1 candidate
11m3	0.4875	0.5952	1630	iter023 session_bridge_merge_top12
12m3	0.4625	0.5852	2057	iter013 querytype_gapelided_v1
13m5	0.4500	0.5642	1743	iter017 contextual_fusion_v1
11m5	0.4375	0.5704	1541	iter017 entity_profiles_guided
13m3	0.4375	0.5633	1791	iter029 session_coverage
12m5	0.4250	0.5815	1822	iter014 adaptive_format
12m7	0.4125	0.5570	1842	iter026 quote_first
13m7	0.4125	0.5196	1886	iter030 trtcer_v1
11m7	0.4000	0.5490	1716	iter025 lead_redundancy

Force-low is cheaper and almost completely closes the reference-history channel: the reference-read share is only 0.1%. Force-high keeps the reference-history channel open, but at substantially higher token cost. This supports the dynamic schedule used by CuraHarness-Full: low budget is useful as a focused, cheap mode, while high budget should be reserved for stages where broader historical evidence is needed.

A.5 Full-context attention ceiling

Table 6 reports how many prior-iteration directories are visible under full-context access and how many of them the proposer actually touches. The budget tier is fixed at high; only the number of visible prior-iteration directories grows with iteration index.

The visible history grows mechanically from about two directories in the first five iterations to about twenty-seven by the end of the run. The number of touched directories grows much more slowly and saturates around three to five. The touch rate therefore falls sharply over time. This supports the bounded-inspection assumption used in Section ??: full-context exposes more history, but the proposer still inspects only a small subset of it.

Table 5: Force-budget ablation for CuraHarness-Full on claudekimi memory runs. Scores are train passrates because these runs diagnose budget behavior rather than final held-out selection. Cost is measured in millions of tokens per proposal.

Benchmark	Policy	Train score	Tokens / prop.	Ref. share
LoCoMo	CuraHarness-Full force-high	0.4000	4.21M	36.4%
	CuraHarness-Full force-low	0.3875	2.79M	0.1%
LongMemEval	CuraHarness-Full force-high	0.5800	3.73M	36.4%
	CuraHarness-Full force-low	0.6300	2.83M	0.1%

Table 6: Full-context exposes a growing prior-iteration pool, but the proposer touches only a small subset. Rows aggregate ten propose iterations from two retained default+direction runs per benchmark.

Benchmark	Iter range	n	Avail. dirs	Dirs touched	Files / touched dir	Ref. reads	Touch rate
LongMemEval	01–05	10	2.0	0.90	5.38	4.50	45%
	06–10	10	7.0	2.00	3.22	5.60	29%
	11–15	10	12.0	2.70	2.40	6.20	23%
	21–25	10	22.0	3.70	2.35	7.50	17%
	26–30	10	27.0	3.50	2.48	7.40	13%
LoCoMo	01–05	10	2.0	1.70	3.04	4.50	85%
	06–10	10	7.0	2.20	3.16	6.10	31%
	11–15	10	12.0	3.80	2.38	8.50	32%
	21–25	10	22.0	3.70	2.25	7.70	17%
	26–30	10	27.0	4.70	2.26	10.50	17%

A.6 Summary

The ablations support four conclusions. First, bounded historical exposure is already useful: Random@3 improves over full-context, showing that an uncured full archive can dilute proposer attention. Second, the best-reference label is the main actionable historical signal. The proposer strongly follows best-tagged directories but rarely inspects worst-tagged directories as negative examples. Third, the best-1 / best-3 reference schedule matches the proposer’s bounded reference attention: one anchor is best for focused exploitation, while three references provide enough comparison at medium budget. Fourth, full-context access increases the number of visible prior iterations much faster than it increases the number of inspected iterations, motivating CuraHarness’s focus on routing attention rather than maximizing exposed history.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction state the paper’s scope as online context curation for automated harness optimization, and the claimed contributions are supported by the method and experimental sections. The claims are limited to the evaluated memory and software-engineering harnesses, with limitations discussed separately.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The paper includes a limitations discussion covering black-box proposer behavior, imperfect control over tool calls and token use, and the limited empirical coverage caused by the cost of automated harness optimization.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not present formal theoretical results or proofs. Its contributions are methodological and empirical.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The experimental setup describes the benchmarks, proposer backends, curation policies, iteration budgets, selection protocol, token accounting, and evaluation metrics needed to reproduce or verify the main comparisons. The released anonymous artifact provides implementation details and scripts for reproduction.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The paper provides an anonymized code repository for submission, including the implementation of CuraHarness and instructions for running the main experiments. The benchmark data are either public or accessed through their standard public interfaces, with preparation details described in the supplemental material.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The experiment section specifies the memory and SWE-bench settings, train/test reporting protocol, iteration budgets, proposer variants, curation tiers, trace scopes, and file-level controller configuration. Additional implementation details are provided in the appendix and released artifact.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: For the memory benchmarks, the paper reports mean and standard deviation over three independent 30-iteration runs. For the more expensive SWE-bench experiment, the paper explicitly reports it as a single 20-iteration run rather than presenting unsupported error bars.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The paper reports token consumption per proposal and describes the compute-relevant experimental budget, including the number of iterations and runs. The supplemental material provides additional execution and resource details needed to reproduce the reported experiments.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work studies automated optimization of agent harnesses on research benchmarks and does not involve deception, private user data, human-subject experiments, or unsafe data release. The submission is anonymized for review.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.

- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The paper discusses the potential benefit of making automated agent optimization more cost-efficient and reproducible, as well as possible risks from improving autonomous coding-agent optimization pipelines without adequate evaluation or oversight.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release a high-risk pretrained model, image generator, or scraped dataset. The released artifact is code for reproducing the harness-optimization experiments.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The paper cites the benchmark datasets, baselines, proposer interfaces, and related harness-optimization systems used in the experiments. The appendix and artifact describe the external assets and their use for reproducibility.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The paper introduces and releases the CuraHarness implementation as a new research artifact. The accompanying anonymous repository documents the code structure, experimental commands, and expected outputs.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.

- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or research with human subjects, so IRB approval or equivalent human-subject review is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLM-based coding-agent proposers are a core component of the studied automated harness-optimization loop. The method and experimental sections describe how these proposers are used and how CuraHarness controls the external evidence workspace shown to them.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.